

Using libraries for creating CAmkES servers

Page Content

- [The technical problem](#)
 - [Requirements](#)
 - [Ideal C-API view for an Application](#)
- [The solutions devised so far](#)
 - [The Crypto Server approach](#)
 - [The File System / Partition Manager approach](#)
 - [The Data61 Timer Server approach](#)
 - [The Configuration Server approach](#)
 - [The TLS Server approach](#)

The technical problem

SEOS functionality usually is implemented by creating libraries. Definition of library in this context: C module offering services which work on a library object. These objects are user provided which means in a given "unit of execution" there may be multiple such library objects existing in parallel. A given SEOS deployment typically will consist of a number of components. Some of these components will want to use SEOS libraries to administrate and use local library instances. Some of them will want to use one or more library instances residing within another component - which in this context is called a server.

Requirements

When thinking about the specifics of how libraries can be implemented fit for local and remote use the following requirements can be identified:

- R1: client component CC has to be able to own and use multiple local instances of library L
- R2: client component CC has to be able to use multiple remote instances of library L which reside in a server component SC
- R3: for a given deployment of SEOS a given client component CC has to be able to use library L as outlined in R1 as well as outlined in R2 in parallel
- R4: the code in CC using library L (= the calls to the library L) should be identical for use cases R1 and R2
- R5: there should be as little code duplication as possible in the implementation of the local and remote use cases for L (C modules, C headers, camkes definition files)

Ideal C-API view for an Application

- C-API is really plain simple API, so it can be used by code written in C or any other programming language that support C bindings
- C-API is implemented in a client glue code layer
- the glue code may have an initialization function that must be called on application start, either explicitly or by a SEOS C-Runtime init.
- the glue code may use dynamic memory allocation, but it should not use a dedicated memory buffer. It should not use malloc() blindly and interfere with the main application
- the C API should have a context parameter that refers to a specific runtime instance of the glue code.
- The application using the C API might not be aware if a function is running locally or in a remote server. Especially, future implementation might change this and there shall be no need to change the application core. Initialization may have to be adopted then, but this could be part of some sandbox code.

The solutions devised so far

The Crypto Server approach

[Unknown User \(carpin01\)](#) please add a succinct description here

The File System / Partition Manager approach

[Unknown User \(chrlin01\)](#) please add a succinct description here

The Data61 Timer Server approach

[Unknown User \(matbuh01\)](#) to provide details

The Configuration Server approach

- There are three modules: "client", "server", "library".
- An interface function takes as its first argument a handle - this is true for all three modules.

- The type of the handle is the same for all three modules.
- A handle can be "local" (in which case it includes a pointer to a library instance) or it can be "remote" (in which case it contains information uniquely identifying a library instance in the server)
- The client and the server own a list of library instances
- User code has to construct a handle before using any library instance be it local or remote.
- For this construction the user code has to provide the information whether the handle should be local or remote plus a library instance identification.
- All user code wanting to use the library end up calling the client.
- The client can deduce if it is allowed to construct the handle or if it has to forward the handle creation request to the server.
- Client and Server create handles by adding identification information to one of their local instances respectively.
- Every library service in the client follows this procedure:
 - if the given handle is a remote handle: call the server implementation
 - if the given handle is a local handle: call the library
- Every library service in the server follows this procedure:
 - if this is a remote handle: transform it into a local handle and call the library
 - else: return with error
- Every library service in the library follows this procedure:
 - if this is a local handle: implement the functionality by using an instance pointer (valid in the current address space)
 - else: return with error
- All three header files and the source modules of the server and library do not require build defines.
- The client source module requires a build define: in case camkes is not used there will be no server instance.
- A client component uses the client and the library module.
- A server component uses the server and the library module.
- The above approach meets requirements R1 to R4.
- For R5 the following can be noted:
 - the header files for client, server and library differ only in the function names and in argument types identifying data ports; this match is so close that it is possible to have these three header files generated from a properly defined "template header file"
 - the signatures in the camkes procedure of the server also uses handles of identical type as their first argument

The TLS Server approach

The TLS API can be configured at run time to run as library ("LIB"), as RPC client ("RPC_CLIENT") and as RPC server ("RPC_SERVER"). It consists of four C files:

1. SeosTlsLib.c – implements the actual functionality
2. SeosTlsRpcClient.c – implements the RPC client side (i.e., calls RPC server)
3. SeosTlsRpcServer.c – implements the RPC server side (i.e. receives RPC Client calls and passes them on to SeosTlsLib.c)
4. SeosTlsApi.c – acts as unified interface for ALL THREE and multiplexes calls internally

It is probably the easiest to illustrate it by showing the code of the initialization:

```
seos_err_t
SeosTlsApi_init(SeosTlsApi_Context*   ctx,
                const SeosTlsApi_Config* cfg)
{
    ctx->mode = cfg->mode;
    switch (cfg->mode)
    {
        case SeosTlsApi_Mode_AS_LIBRARY:
            return SeosTlsLib_init(&ctx->context.library, &cfg->config.library);
        case SeosTlsApi_Mode_AS_RPC_CLIENT:
            return SeosTlsRpcClient_init(&ctx->context.client, &cfg->config.client);
        case SeosTlsApi_Mode_AS_RPC_SERVER:
            return SeosTlsRpcServer_init(&ctx->context.server, &cfg->config.server);
        default:
            return SEOS_ERROR_NOT_SUPPORTED;
    }
    return SEOS_ERROR_GENERIC;
}
```

Now, there is a distinction for the other calls to the TLS API – those which implement actual functionality, which is again best shown in an example: This code shows how a call to SeosTlsApi_init() is internally mapped to the respective sub-module and that each sub-module can maintain its own context and configuration as part of the generic context/configuration.

```

seos_err_t
SeosTlsApi_handshake(SeosTlsApi_Context*   ctx)
{
    switch (ctx->mode)
    {
        case SeosTlsApi_Mode_AS_LIBRARY:
            return SeosTlsLib_handshake(&ctx->context.library);
        case SeosTlsApi_Mode_AS_RPC_CLIENT:
            return SeosTlsRpcClient_handshake(&ctx->context.client);
        default:
            return SEOS_ERROR_NOT_SUPPORTED;
    }
    return SEOS_ERROR_GENERIC;
}

```

The definition of the SeosTlsApi_Context and SeosTlsApi_Config reflect similarly how the API module/context/config is simply a wrapper/multiplexer for SeosTlsLib.c/SeosTlsRpcClient.c/SeosTlsRpcServer.c

```

struct SeosTlsApi_Context
{
    SeosTlsApi_Mode          mode;
    union
    {
        SeosTlsLib_Context   library;
        SeosTlsRpcClient_Context client;
        SeosTlsRpcServer_Context server;
    } context;
};

typedef struct
{
    SeosTlsApi_Mode          mode;
    union
    {
        SeosTlsLib_Config    library;
        SeosTlsRpcClient_Config client;
        SeosTlsRpcServer_Config server;
    } config;
} SeosTlsApi_Config;

```

The component acting as RPC_SERVER thus has to include SeosTlsRpcServer.c as part of the compilation process. which also needs a CaMKES interface definition which maps to the SeosTlsRpc_xxx() functions called from the RPC_CLIENT. If the API is running as LIB or RPC_CLIENT, then these calls can be passed to the respective module. However, if the API runs as RPC_SERVER, it has to be part of a distinct component and calls will come through another instance of the API configured as RPC_CLIENT.

This is how the SeosTlsRpcServer_handshake() implementation looks like:

```

seos_err_t
SeosTlsRpcServer_handshake(SeosTlsRpcServer_Handle handle)
{
    SeosTlsRpcServer_Context* ctx = &handle->context.server;
    if (handle->mode != SeosTlsApi_Mode_AS_RPC_SERVER)
    {
        return SEOS_ERROR_OPERATION_DENIED;
    }

    return SeosTlsLib_handshake(&ctx->library);
}

```

The crucial thing to note is that SeosTlsRpcServer_Handle is actually a typedef for SeosTlsApi_Context*.

Evaluation w.r.t. the requirements:

- R1: Easily doable by initializing and using parallel API contexts configured as lib.
- R2: Easily doable by initializing and using parallel API contexts configured as RPC client (just needs an equal amount of RPC server instances distributed across other components – or in a single component).
- R3: Easily doable by initializing API contexts as lib/RPC client and using them in the respective context.

- R4: Fulfilled.
- R5: There is a "somewhat" duplication of function signatures for functionality provided by SeosTlsApi.c, SeosTlsLib.c and SeosTlsRpcClient.c because essentially SeosTlsApi.c multiplexes function calls between the other two. The definition of the Camkes interface is again a slight duplication of this functionality, because, obviously it needs to match what is offered by SeosTlsLib.c